

7. Tvorba konkrétních metod

V této kapitole si zkusíme vytvořit pár jednoduchých metod pracujících s objekty typu `Zak`.

Metoda: Změň příjmení

První metodu nazveme **`zmenPrijmeni`**. Jelikož vytváříme systém hodící se i pro střední školství, je možné, že žákyně by se během studia mohla vdát a následně i změnit příjmení. Dává tedy smysl metodu naprogramovat.

Přemýšlejme, co do metody bude vstupovat. Proměnná `prijmeni` nikoli, neboť ji sice měníme, ale již ji máme k dispozici v objektu. Nemá tedy smysl, aby do metody vstupovala přes parametr. Budeme ale potřebovat nové příjmení, kterým chceme staré přepsat. Do metody tedy bude vstupovat textový řetězec `String`, který pojmenujeme `novePrijmeni`.

Výstup nebude mít metoda žádný. Dle zadání nepožadujeme, aby metoda něco vrátila či zobrazovala. Má jen změnit hodnotu proměnné `prijmeni`, toť vše. Výstup tedy nastavíme jako **`void`**.

Metoda bude, tak jako všechny dosud konstruované metody, veřejná (**`public`**), abychom ji mohli zavolat z jiné třídy. Konkrétně ze třídy `Main.java`. Programování bude relativně snadné.

Metoda `zmenPrijmeni` (`Zak.java`)

```
public void zmenPrijmeni(String novePrijmeni)
{
    prijmeni = novePrijmeni;
}
```

Volání metody `zmenPrijmeni` (`Main.java`)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001,Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana","Výborná","115111/5284",ld2, false,(byte)2);
System.out.println(zak2.vratInfo());
zak2.zmenPrijmeni( novePrijmeni: "Chvalitebná");
System.out.println(zak2.vratInfo());
```

Run: Main ×

↑	Jana Výborná (20. 3. 2001), 2. ročník
↓	Jana <u>Chvalitebná</u> (20. 3. 2001), 2. ročník

Metoda: Zvyš ročník

Další metoda **zvysRocnik** bude mít za úkol postoupit žáka do následujícího ročníku.

Ročník dle zadání zvyšujeme vždy o číslo 1, nemá tedy smysl, aby číslo ročníku bylo vstupem. Postačuje nám původní číslo ročníku již uložené v samotném objektu. Do metody nebude vstupovat nic. Bude tzv. **bezparametrická**. Nesmíme ale zapomenout napsat za název metody **prázdné závorky**.

Výstupem nebude rovněž nic. Metoda proměnnou pouze modifikuje. V zadání nebylo o zpětném výpisu ani slovo. Výstup tedy v hlavičce označíme jako **void**.

Nezapomeneme opět připsat modifikátor přístupu **public**.

Metoda zvysRocnik (Zak.java)

```
public void zvysRocnik()
{
    rocnik++;
}
```

Dále je zobrazeno volání metody `zvysRocnik` a rovněž důkaz správného fungování v podobě volání metody `vratInfo()`.

Volání metody zvysRocnik (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.vratInfo());
zak2.zvysRocnik();
System.out.println(zak2.vratInfo());
```



```
Run: Main x
Jana Výborná (20. 3. 2001), 2. ročník
Jana Výborná (20. 3. 2001), 3. ročník
```

Metoda: Pohlaví

Další veřejná metoda nazvaná **pohlavi** nám bude vracet pohlaví žáka. Na základě stavu proměnné `chlapec` vrátíme za `true` řetězec „chlapec“ a za `false` řetězec „dívka“.

Do metody tedy nebude vstupovat žádná hodnota, neboť proměnnou `chlapec` už má objekt k dispozici a žádnou jinou pro vyhodnocení pohlaví nepotřebuje.

Z metody bude vystupovat text určující pohlaví žáka, tedy objekt typu `String`.

Mějme na paměti, že pokud v hlavičce diktujeme výstup, v těle metody musí tato výstupní hodnota:

- být označena klíčovým slovem **return**
- existovat za každých okolností – není možné v těle metody vynechat větev else. Za určitých okolností by totiž výstup neexistoval a kompilátor by Vám dal jasně najevo, že se mu tento stav nelíbí.

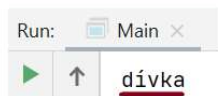
Metoda pohlavi (Zak.java)

```
public String pohlavi()
{
    if (chlapec)
        return "chlapec";
    else
        return "dívka";
}
```

Následuje volání metody. Jelikož samotná metoda neřeší výpis textu na konzoli (což lze považovat za optimální řešení), nesmíme opomenout výstup zobrazit ve třídě Main.java pomocí příkazu System.out.println.

Volání metody pohlavi (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.pohlavi());
```



Metoda: Plnoletost

Poslední metoda bude nejobtížnější. K realizaci potřebujeme znalost práce s třídou **LocalDate**. Metoda **plnoletost** nám vrátí pravdivostní hodnotu true/false dle proměnné **datumNarozeni**.

Datum narození žáka má objekt k dispozici, nebude tedy vstupem. Potom už potřebujeme k výpočtu pouze aktuální datum. To lze ovšem jednoduše zjistit během vykonávání metody. Vstupy tedy nemusíme řešit.

Výstupem rozumíme true, pokud je žák plnoletý a false, pokud plnoletý není. Na místo výstupu tedy uvedeme datový typ boolean.

Tělo metody není složité, ale žádá si určité znalosti týkající se práce s časovými intervaly. Pro správné fungování je nezbytné naimportovat v horní části třídy Zak příslušné balíčky!

Metoda plnoletost (Zak.java)

```
import java.time.LocalDate;
import java.time.Period;
import java.time.temporal.ChronoUnit;

public class Zak {

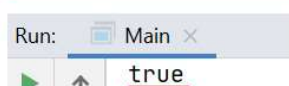
    public boolean plnoletost()
    {
        LocalDate dnes = LocalDate.now();
        Period doba = Period.between(datumNarozeni, dnes);
        int vek = (int)doba.get(ChronoUnit.YEARS);

        if (vek >= 18)
            return true;
        else
            return false;
    }
}
```

Volání metody z Main.java je již pro nás rutinou. Vypisujeme na konzoli výstup, který nám metoda vrátí. V našem případě je žákyně již plnoletá.

Volání metody plnoletost (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.plnoletost());
```



V příští kapitole si ukážeme metody typické pro programy konstruované v Javě, metody get a set.

Zadání

Přepište tělo metody pohlavi() pomocí jednoduchého ternárního operátoru. Pokud nevíte, co to ternární operátor je, nejprve si látku samostatně nastudujte.

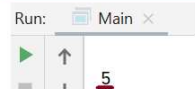
Popřemýšlejte, jak by se musela změnit metoda zvysRocnik(), když bychom chtěli ošetřit, aby ročník nemohl být vyšší než maximální povolená hodnota.

8. Modifikátory přístupu, Gettry a Settry

Při konstrukci třídy Zak jsme veškeré naše proměnné označili **modifikátorem přístupu public**. To znamená, že vlastnosti žáka jsou **veřejné**. Jak **pro zápis**, tak **pro čtení**. Lze je přepisovat či zobrazovat z kterékoli části programu. Práce s proměnnými objektu je triviální - pomocí tečky za konkrétním objektem se k proměnné vždy jednoduše dostaneme.

Použití tečkové konvence při práci s public atributem

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
zak2.rocnik=5;
System.out.println(zak2.rocnik);
```



Toto zjednodušení se však může záhy obrátit v nevýhodu. Co když díky nepozornosti během programování umožníme z úplně jiného místa programu modifikaci proměnné, kterou jsme nezamýšleli? Nebo tuto modifikaci někdo provede záměrně, protože prostě může. Změna jména žáka by byla lehce dohledatelná a opravitelná. Kdybychom ale například do programu přidali možnost známkování, změna konkrétních hodnocení by se napravovala velice těžko. Navíc program by přestal být důvěryhodný a v praxi by jej nechtěl nikdo používat. Řešením je právě změna modifikátoru přístupu public. Podívejme se, jaké máme možnosti.

Typy modifikátorů přístupu u proměnných či metod

Public

Proměnná/metoda je dostupná z kterékoli třídy programu.

Private

Proměnná/metoda je dostupná pouze ve třídě, ve které se nachází.

Implicitní – bez použití modifikátoru přístupu

Proměnná/metoda je dostupná pouze z těch tříd, které jsou ve stejném balíčku.

Protected

proměnná/metoda je dostupná pouze z těch tříd, které jsou ve stejném balíčku, nebo z potomků dané třídy (tyto podděděné třídy se mohou nacházet i v jiných balíčcích).

Poznámka: Předpokladem je dědičnost. Použití si ukážeme v kapitole č. 13.

Pojďme tedy naše proměnné z class Zak skryt – uzavřít je před zbytkem programu. Nastavme jim modifikátor přístupu **private** a zkoumejme, co se stane.

Změna modifikátorů přístupu u proměnných ve třídě Zak (Zak.java)

```
public class Zak {  
    private String jmeno;  
    private String prijmeni;  
    private String rodneCislo;  
    private LocalDate datumNarozeni;  
    private boolean chlapec;  
    2 related problems  
    private byte rocnik;
```

Poznámka: Chyba viditelná výše je způsobena starým kódem zapsaným v Main.java, kdy pro práci s proměnnou rocnik ještě používáme tečkovou konvenci. Nyní už totiž třída Main na proměnnou rocnik nevidí. Smazání chybového kódu ze třídy Main nám vyřeší i chyby ve třídě Zak.

Chybový kód ve třídě Main (Main.java)

```
//zak2  
LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);  
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);  
zak2.rocnik=5;  
System.out.println(zak2.rocnik);
```

Poznámka: Pokud Vám z dřívějšíka zůstal ve třídě Main ještě objekt zak1, tento objekt rovněž od této chvíle tvoří chybu a musí být celý smazán.

Nyní je veškerá přímá práce s proměnnými zakázaná. Jediná možnost, jak se k nim dostat je pomocí **neprivátních** metod. Tedy každá požadovaná operace s proměnnými bude vyžadovat novou vlastní metodu. Abychom nemuseli nad problematikou příliš hloubat, budeme považovat neprivátní metodu vždy za **public** metodu. Ostatně, pár public metod už ve své třídě máme..

Aktuálně máme z třídy Main.java dovolenu:

- Tvorbu (zápis) nových objektů pomocí metod - konstruktorů
- Hromadné čtení atributů pomocí metod toString a vratInfo
- Zápis do proměnné prijmeni pomocí metody zmenPrijmeni

Metody Get a Set

Na jakémkoli modifikátoru přístupu je nepříjemné, že u dané proměnné vždy **určuje zároveň jak práva pro čtení, tak práva pro zápis**. Co kdybychom ale chtěli vyrobit proměnnou, kterou bude možno číst, ale zároveň do ní nebude možno z logických či bezpečnostních důvodů zapisovat?

Řešení je snadné – pokud potřebujeme neveřejnou proměnnou číst, vyrobíme pro tyto účely metodu označenou jako **get**. Pokud je naším cílem do proměnné zapisovat, vyrobíme metodu označenou jako **set**.

Pravidla tvorby názvů metod get a set jsou ustálená a velmi jednoduchá:

- Pro settery – zápis: setNazevPromenne
- Pro gettery - čtení: getNazevPromenne
- Pro getter typu boolean: isNazevPromenne

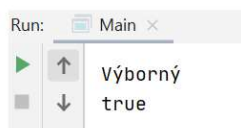
Metody get a set u privátních proměnných příjmení, chlapec (Zak.java)

```
public class Zak {  
    private String jmeno;  
    private String prijmeni;  
    private String rodneCislo;  
    private LocalDate datumNarozeni;  
    private boolean chlapec;  
    private byte rocnik;  
  
    public void setPrijmeni(String prijmeni){  
        this.prijmeni=prijmeni;  
    }  
  
    public String getPrijmeni(){  
        return prijmeni;  
    }  
  
    public boolean isChlapec(){  
        return chlapec;  
    }  
  
    public void setChlapec(boolean chlapec){  
        this.chlapec = chlapec;  
    }  
}
```

Poznámka: Pokud Vám nyní přijde zbytečná dříve programovaná metoda zmenPrijmeni, máte pravdu, klidně ji vymažte. SetPrijmeni Vám udělá stejnou službu.

Volání metod get a set (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Vlasta", "Výborná", "115111/5284", ld2, false, (byte)2);
zak2.setPrijmeni("Výborný");
zak2.setChlapec(true);
System.out.println(zak2.getPrijemni());
System.out.println(zak2.isChlapec());
```



Každý atribut Vaší třídy musíte podrobit pečlivé rozvaze pro určení vhodných přístupových práv. V případě příjmení dává smysl vytvořit jak getter, tak setter. Kdyby ale žák disponoval například proměnnou pro uložení průměru všech známek, jistě by se tato hodnota vypočítávala z žákových dosavadních výsledků. Použit zde setter pro přímý přepis průměru by nebylo rozumné. Z žáka nevalných výsledků by se mohl rázem stát premiant. Metoda get by byla pro zjištění hodnoty průměru naprosto v pořádku. Proměnnou průměr bychom tedy z hlediska logiky programovali pouze pro čtení.

Je vždy nutné zamyslet se nad skutečností, zda změna hodnoty jedné proměnné by neměla zapříčinit změnu hodnoty jiné proměnné programu. Zda existuje nějaká logická souvislost či vazba proměnných mezi sebou. Změna známky ovlivní celkový průměr žáka. Rodné číslo má závislost na datu narození a pohlaví. Číslo ročníku má souvislost s konkrétní třídou, ve které se žák nachází. Při dovršení 18ti let se žák stává svým vlastním zákonným zástupcem... takových spojení může být ve vašich programech celá řada a je nutné si je pokud možno všechny ohlídat. K implementaci těchto vazem Vám jistě poslouží právě settry. V těle metody set nemusíte pouze měnit hodnotu daného atributu. Lze zde rovněž zabezpečit vazby na ostatní části programu. Předejdete tak hromadě logických nesmyslů, které by Váš program jinak umožnil.

Vyvarujme se používání modifikátoru přístupu public u našich atributů. Metody get a set nám udělají mnohem lepší službu. Přiznávám, že programování bude o dost pracnější. Zároveň však tímto přístupem získáme plnou kontrolu nad tím, jak se s našimi důležitými daty nakládá.

Zadání

Podívejte se na veškeré proměnné třídy Zak.java. Rozvažte, kdy a za jakých podmínek byste u té či oné proměnné dopsali právě getter či setter.

9. Třída ArrayList (teorie)

Třidu Zak máme z podstatné části nadefinovanou. Ve třídě Main.java nyní můžeme vytvořit libovolné množství objektů – instancí třídy Zak. Pojmenujeme je zak1 až zak1000... ne, toto určitě není vhodný postup ukládání většího množství objektů. Použijeme raději kontejner. Nebude se ovšem jednat o klasické pole, nýbrž o vylepšený objekt typu ArrayList. Oproti poli můžeme u tohoto kontejneru dynamicky měnit počet prvků. Nadefinované metody nám například dovolí snadno vložit nový prvek doprostřed listu či jej z listu odstranit.

Zde uvádím příklad deklarace ArrayListu v metodě main třídy Main.java. Jedná se o tzv. **generický kontejner** – v jeho definici uvádíme, co přesně do něj chceme ukládat. V našem případě se nejedná o celá čísla, ani textové řetězce, ale přímo o objekty typu Zak.

Pozor, **ArrayList** je specifický v tom, že **ukládá pouze referenční datové typy**, nikoli primitivní datové typy. Tedy můžeme ukládat objekty typu Zak, String, ..., ale není možné ukládat např. hodnoty datového typu int. To ale neznamená, že zde nelze ukládat celá čísla. Má to jen jeden háček. Musíme hodnotu int vydávat za objekt typu **Int**. Tento proces, kdy v podstatě primitivní datový typ balíme do nadřazené třídy Objekt, nazýváme **boxing**.

Tvorba ArrayListu

Nyní přejdeme k samotnému programování. Nezapomeneme importovat **Java.util.ArrayList**.

Deklarace objektu typu arrayList (Main.java)

```
import java.util.ArrayList;
public class Main {

    static ArrayList<Zak> zaci = new ArrayList<Zak>();

    public static void main(String[] args) {
```

Poznámka: Všimněte si klíčového slova static u deklarace ArrayListu. Stejné slovo se nachází i u hlavičky metody main. Tento modifikátor značí, že se v rámci programu jedná o jediný exemplář ArrayListu.

Žáky v metodě main snadno přidáme pomocí předdefinované metody **add**. Ta vždy přidává nový prvek na konec seznamu. Každým voláním metody add tedy zvětšujeme počet íprvků seznamu o 1.

Různé způsoby deklarace žáka a jeho uložení do seznamu (Main.java)

```
public static void main(String[] args) {  
  
    LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);  
    Zak zak1 = new Zak("Vlasta", "Výborná", "115111/5284", ld2, false, (byte)2);  
    zaci.add(zak1);  
  
    Zak zak2 = new Zak("Jindřich", "Ospalý", "981129/1235", LocalDate.of( year: 2002, month: 12, dayOfMonth: 14), true);  
    zaci.add(zak2);  
  
    zaci.add(new Zak("Petr", "Poškořák", "123456/1234", LocalDate.of( year: 2002, month: 4, dayOfMonth: 20), true));  
}
```

Poznámka: Poslední způsob je nejkratší. Má též výhodu, že nemusíme řešit pojmenování instance objektu, jako např. zak100.

Procházení ArrayListu

Abychom si ověřili, že došlo k zápisu objektů do RAM paměti, celý seznam si projdeme index po indexu pomocí cyklu **for**. V těle cyklu pak na každé z instancí zavoláme metodu `vratInfo()`. Na konzoli se objeví příslušné informace o třech uložených žácích.

V záhlaví cyklu voláme metodu **size()**, která vrací počet prvků seznamu. Metoda **get** nám umožní přečíst ze seznamu položku uloženou na daném indexu.

Procházení seznamu pomocí cyklu for

```
for (int i = 0; i < zaci.size(); i++) {  
    System.out.println(zaci.get(i).vratInfo());  
}
```

Jednodušší způsobem pro průchod našeho seznamu je cyklus **foreach**. Při použití tohoto cyklu se vůbec nestaráme o indexaci (řídící proměnnou *i*). Tento cyklus vždy striktně předpokládá pohyb od 0. tého k poslednímu prvku. Indexace je implementována do jeho vnitřní logiky a my už ji tím pádem nemusíme programovat. Stačí nám určit zástupný název pro objekt třídy `Zak`, zde jsem zvolil malé *z*.

Procházení seznamu pomocí cyklu foreach

```
for (Zak z: zaci) {  
    System.out.println(z.vratInfo());  
}
```

Výstupy na konzoli jsou při použití obou cyklů totožné, dané tělem metody **vratInfo()**.

Výpis seznamu na konzoli

```
Run: Main ×
Vlasta Výborná (20. 3. 2001), 2. ročník
Jindřich Ospalý (14. 12. 2002), 1. ročník
Petr Poškořák (20. 4. 2002), 1. ročník
```

Jak vidíte, používání seznamu k ukládání objektů je jednoduché a efektivní.

Předdefinované metody ArrayListu

Zde příkládám seznam často používaných metod, které má třída ArrayList k dispozici.

Přidávání prvků:

- `add(prvek)` – na konec seznamu
- `add(index, prvek)` – kdekoli do seznamu
- `addAll(konekce prvků)` – více prvků najednou

Odstraňování prvků:

- `remove(prvek)` – **odstraní** prvek, ale pozor, **pouze jeho první výskyt!**
- `remove(index)` – odstraní prvek na daném indexu
- `removeAll(index, počet)` – odstraní daný počet prvků od daného indexu
- `clear()` – odstraní všechny prvky

Bystřejší z Vás jistě zaznamenali možný zádrhel. Pokud bychom měli v ArrayListu uložen seznam intergerů a chtěli bychom použít metodu `remove`, co by znamenalo číslo na vstupu? Prvek, nebo index? Logika je nastavena způsobem, že se smaže daný index (aktuální hodnotu na indexu). Pokud bychom chtěli vyhledat a smazat konkrétní celočíselnou hodnotu (respektive její první výskyt v seznamu), nejjednodušším způsobem je zápis `remove(new Integer(prvek))`. Jedná se o dříve vysvětlený **boxing**, kdy celé číslo bereme jako objekt typu Integer.

Diagnostika:

- `contains(prvek)` – vrátí hodnotu `true/false` dle toho, zda se daný prvek v seznamu vyskytuje či nikoli.
- `indexOf(prvek)` – vrátí pořadové číslo prvního indexu, na kterém se hledaný prvek nachází. V případě nenalezení prvku vrací metoda hodnotu **-1**.

- `lastIndexOf(prvek)` – vrátí pořadové číslo posledního indexu, na kterém se hledaný prvek nachází.
V případě nenalezení prvku vrátí metoda rovněž hodnotu -1

Na závěr ještě zmíním metodu **`toArray`**, která je schopna převést `ArrayList` na klasické pole. Někdy se může stát, že metody, které chceme používat, na svém vstupu tolerují pouze `Array`. Potom se Vám tato metoda bude jistě hodit.

Převod ArrayListu na objekt typu Array

```
Zak[] zaciPole = new Zak[zaci.size()];  
zaciPole = zaci.toArray(zaciPole);
```

V příští kapitole se pokusíme `ArrayList` zakomponovat do našeho programu vhodnějším způsobem. Budeme tvořit objekty typu `Trida`.

10. Class Třída, metody pro výpis

Když přemýšlíme nad strukturou objektově orientovaného programu, je záhodno si nejprve vyčlenit, ze kterých objektů se bude náš program skládat. Jedna z pomůcek je říci si věty jednoduše popisující programovaný systém. Potom z těchto vět vyčlenit podstatná jména a z těch pak vytvořit naše třídy.

Program Evidence školy má za úkol v jedné škole spravovat jednotlivé třídy. Ve škole pracují učitelé a nepedagogičtí pracovníci. Ve třídách sedí žáci. Žáci jsou zapsáni na určité obory, dle nich pak studují konkrétní předměty. Na základě svých výsledků dostávají známky. Zároveň se jim počítá absence..

Vidíte, že náš program se tímto způsobem docela rychle nafukuje. Vyčlenili jsme classy: Skola, Trida, Obor, Predmet, Zak, Pedagogicky_pracovnik, Nepedagogicky_pracovnik, Znamkovani, Absence..

V této učebnici budeme vytvářet pouze jednu dílčí část programu pro evidenci školy. Kdybychom chtěli řešit kompletní program, učební text by se stal dlouhým a také již docela nepřehledným. Základní principy jednoduchých programů však platí i pro ty složitější, proto na konci učebnice byste v případě zájmu měli být schopni celý program dořešit samostatně.

Definování class Trida

K našemu zjednodušení – předpokládejme, že v celé škole máme pouze jednu třídu. Nikoli ArrayList plný tříd, ale jedinou třídu.

Taková třída obsahuje určité seznam žáků, dále číslo ročníku a třeba ještě jméno třídního učitele. Jistě Vás napadají další atributy, pro zestručnění programu je však nyní vynecháme.

Atributy (Trida.java)

```
package BalicekProjektu;
import java.util.ArrayList;

public class Trida {
    private ArrayList<Zak> zaci;
    private byte rocnik;
    private String tridni;
}
```

Třída bude mít na začátku seznam žáků prázdný. Proto postačí jej vytvořit v těle konstruktoru.

Konstruktory (Trida.java)

```
public Trida(byte rocnik, String tridni)
{
    this.rocnik = rocnik;
    this.tridni = tridni;
    zaci = new ArrayList<Zak>();
}
```

Metody vypisující informace o třídě

Nyní naprogramujeme metody pro výpis. Začneme známou metodou **toString**, která bude vypisovat vnitřní stavy atributů třídy *Trida*. Nezapomeneme použít klíčové slovo **override**. Metodu vygenerujeme automaticky pomocí klávesové zkratky **levý Alt + Insert**.

Metoda toString (Trida.java)

```
@Override
public String toString() {
    return "Trida{" +
        "zaci=" + zaci +
        ", rocnik=" + rocnik +
        ", tridni='" + tridni + '\'' +
        '}';
}
```

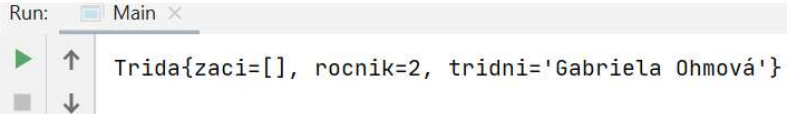
Při zakládání nového objektu klasicky zavoláme konstruktor. O výpis objektu se postará před chvílí programovaný **toString**. Připomínám, že **System.out.println** volá metodu **toString** implicitně. Není tedy potřeba název metody v kódu uvádět.

Instance třídy a výpis pomocí metody toString (Main.java)

```
public class Main {

    public static void main(String[] args) {

        Trida t1 = new Trida((byte)2, "Gabriela Ohmová");
        System.out.println(t1);
    }
}
```



Po další metodě budeme chtít vypsat informace o všech žácích v dané třídě. Pojmenujeme ji **VratSeznamZaku**. Do metody nic nebude vstupovat, neboť všechny informace pro výpis má třída k dispozici. Výstupem je samozřejmě **String**.

V metodě procházíme pomocí cyklu **foreach** veškeré žáky v seznamu a voláme na nich jejich vlastní metodu **vratInfo**. Díky provázanosti objektů jsme tedy schopni z jedné třídy volat metodu úplně jiné třídy. Tedy pokud je **public**. Velmi nám to usnadňuje práci, neboť formát pro výpis nemusíme v kódu znovu definovat. Celý výpis pro přehlednost uzavřeme do textové grafiky.

Metoda `vratSeznamZaku` (`Trida.java`)

```
public String vratSeznamZaku()
{
    String text="";
    text+="*****Seznam žáků*****"+System.lineSeparator();
    for(Zak z:zaci)
        text+=z.vratInfo()+System.lineSeparator();
    text+="*****"+System.lineSeparator();
    return text;
}
```

Poznámka: Metodu nevyzkoušíme, neboť nejsme schopni vložit do našeho seznamu žáků testovací data. Proč? Neboť samotný `ArrayList` je `private`. Mohli bychom jej udělat veřejný nebo pro něj vytvořit metodu `Set`. Testování si ale necháme na příště, kdy budeme vytvářet metodu cílenou přímo na přidávání žáků.

Poslední metoda s názvem **`vratInfo`** bude vracet kompletní informace o naší třídě, tedy včetně ročníku a třídního. Abychom si programování zjednodušili, zakomponujeme do kódu předchozí metodu `VratSeznamZaku`.

Metoda `vratInfo` (`Trida.java`)

```
public String vratInfo()
{
    String text=System.lineSeparator()+"*****"
        +System.lineSeparator();
    text+=String.format("%d. třída (třídní: %s)%n", rocnik, tridni);
    text+=vratSeznamZaku();
    return text;
}
```

V příští kapitole si zapřemýšlíme nad dalšími metodami vhodnými pro práci se třídou. Začneme dříve zmiňovanou metodu **`pridejNovehoZaka`**.

Zadání

Pokuste se pro potřeby class `Trida` vytvořit metodu, která bude vracet očíslovaný jmenný prostor žáků. Číslování bude začínat od 1. Tedy výstup bude např.:

Výborný Vlasta

Ospalý Jindřich

Poškolák Petr ...

11. Další metody pro class Třída

V této kapitole si naprogramujeme pár metod pro naši novou classu Třída. Vždy je nutné podrobit pečlivé rozvaze, jaké metody chceme programovat a co přesně budou vykonávat. Je možné, že zde popsane „podprogramy“ byste řešili jiným způsobem. Záleží na zadání, které jste dostali od klienta. Jaké požadavky přesně na třídu budou kladeny, podle čeho se má metoda rozhodovat atd... následující kapitola popisuje jen pár možností, jak se třídou pracovat.

V minulé kapitole jsem slíbil začít metodou **pridejNovehoZaka**, která má za úkol přidat nového žáka do seznamu. Pojdme o ní zapřemýšlet.

Nejjednodušší postup:

- Vytvoříme nového žáka pomocí konstruktoru ještě před voláním metody **pridejNovehoZaka**.
- Instance žáka bude vstupem to metody: **pridejNovehoZaka(Zak z)**.
- V těle metody pouze vložíme žáka do seznamu pomocí metody **add**: **zaci.add(z);**.

Kód by fungoval, co když ale žák ve třídě již existuje? Pak by byl ve třídě 2x což jistě není žádoucí. V tom případě bychom žáka do seznamu naopak přidávat nechtěli. Přibyl nám tedy další problém – jak reálně ověřit, že žák v seznamu již je? Naskýtají se nám 3 možnosti:

1. V metodě nejprve vytvoříme žáka z poskytnutých údajů a následně se dotazujeme seznamu, zda tento objekt již v seznamu je.
2. Vytvoříme objekt typu Zak. Následně z něj pomocí existující metody **hashCode()** vytvoříme jednoznačný identifikátor, tzv. **hash**. Při prohledávání seznamu žáků tak porovnáme pouze identifikátory.
3. Před samotnou tvorbou objektu projdeme seznam žáků a srovnáme pouze určité atributy žáka, které si zvolíme. Například můžeme spoléhat na to, že kombinace jména, příjmení a roku narození má minimální šanci se u více žáků zopakovat. Objekt typu Zak tedy vytváříme až tehdy, kdy to má smysl - až po ověření, že v seznamu není přítomen.

Za mě jsem si zvolil poslední možnost. K porovnávání studentů budu používat jediný atribut. Jednoznačný identifikátor – **rodné číslo**.

Metoda: Je žák ve třídě

Je zřejmé, že přítomnost žáka ve třídě budu v programu řešit opakovaně. Logicky si tedy vytvořím novou metodu. Nazveme ji **jeZakVeTride**.

Kritéria metody jeZakVeTride:

- Vstupem metody bude hledané rodné číslo, tedy String. ArrayList reprezentující seznam existujících žáků do metody vstupovat nebude. Už je součástí objektu a přidávání na vstup by tedy nedávalo smysl.
- Z metody bude vystupovat true/false, tedy žák ve třídě je/není.
- Metodu jsem se rozhodl označit modifikátorem přístupu **private**. Z pohledu programu to znamená, že metodu nebude možné zavolat z jiné classy než Trida. V podstatě vytváříme podprogram pouze pro potřeby této třídy.

Metoda prochází seznam a hledá shodu v rodném čísle. Pozor, předpokladem fungování je správné nadefinování metody **getRodneCislo** ve třídě Zak.

Metoda jeZakVeTride (Trida.java)

```
public boolean jeZakVeTride(String rodneCislo)
{
    for (Zak z:zaci) {
        if (z.getRodneCislo().equals(rodneCislo))
            return true;
    }
    return false;
}
```

Jak je z kódu patrné, v podmínce pro porovnání nepoužíváme operátor `==`, nýbrž metodu **equals**. Uvědomme si, že v podstatě porovnáváme objekty – textové řetězce. Tedy použití equals je na místě.

Metoda: Přidej nového žáka

Metoda **pridejNovehoZaka** bude mít za úkol následující:

- Převezme na vstupu všechny hodnoty potřebné pro tvorbu nového žáka.
- Dle rodného čísla zjistí, zda se již žák vyskytuje v seznamu či nikoli.
- Pokud se žák v seznamu nevyskytuje, vytvoří nového žáka.
- Posléze jej přidá do seznamu žáků.

Do metody **pridejNovehoZaka** budou vstupovat veškeré atributy potřebné pro tvorbu žáka (pro volání konstrukturu třídy Zak). Tedy až na jeden, a to ročník. Ten se doplní automaticky dle třídy, do které žáka

přidáváme. Pokud budu žáka přidávat do třídy, která má v atributu `rocnik` uloženu hodnotu 3, automaticky se v těle metody tato hodnota 3 přepíše do atributu `rocnik` u žáka.

Metoda `pridejNovehoZaka` (`Trida.java`)

```
public void pridejNovehoZaka(String jmeno, String prijmeni, String rodne_cislo, LocalDate datum_narozeni, boolean chlapec)
{
    if (!jeZakVeTride(rodne_cislo))
        zaci.add(new Zak(jmeno, prijmeni, rodne_cislo, datum_narozeni, chlapec, rocnik));
}
```

Abychom pro uživatele poskytli základní zpětnou vazbu, doplníme metodu o vypisování informací o stavu metody na konzoli. Uživatel bude mít přehled, co se v programu právě děje. Návratovou hodnotou metody tedy bude `String`.

Metoda `pridejNovehoZaka` s výstupem (`Trida.java`)

```
public String pridejNovehoZaka(String jmeno, String prijmeni, String rodne_cislo, LocalDate datum_narozeni, boolean chlapec)
{
    if (!jeZakVeTride(rodne_cislo))
    {
        zaci.add(new Zak(jmeno, prijmeni, rodne_cislo, datum_narozeni, chlapec, rocnik));
        if (chlapec)
            return String.format("Žák %s %s byl přidán do %d. třídy", jmeno, prijmeni, rocnik);
        else
            return String.format("Žákyně %s %s byla přidána do %d. třídy", jmeno, prijmeni, rocnik);
    }
    else
        return String.format("Chyba přidání nového žáka: Bylo zjištěno, že žák/žákyně s rodným číslem %s již ve třídě existuje.", rodne_cislo);
}
```

Pojďme si nyní funkčnost metody `pridejNovehoZaka` vyzkoušet. Nejprve ve třídě `main` vytvoříme novou třídu `t1`. Následně do ní přidáme 3 různé žáky. Úspěšné přidání můžeme kontrolovat na konzoli jak během samotného volání metody, tak během volání metody `vratSeznamZakuz` na objektu `t1`.

Testování metody `pridejNovéhoZaka` (`Main.java`)

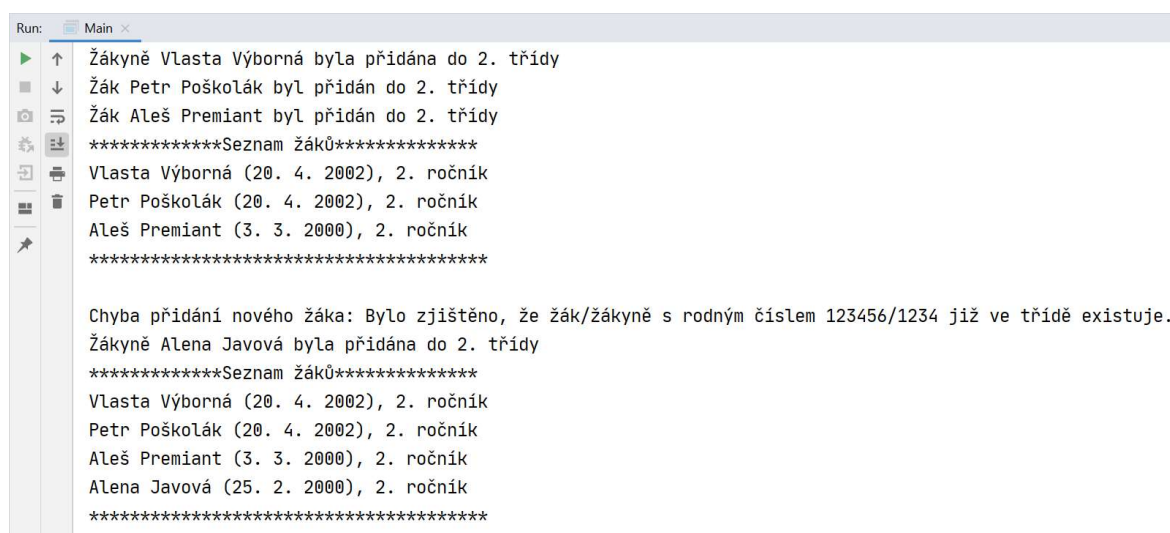
```
public static void main(String[] args) {
    Trida t1=new Trida((byte)2,"Gabriela Ohmová");

    System.out.println(t1.pridejNovéhoZaka( jmeno: "Vlasta", prijmeni: "Výborná", rodne_cislo: "987654/3210",
        LocalDate.of( year: 2002, month: 4, dayOfMonth: 20), chlapec: false));
    System.out.println(t1.pridejNovéhoZaka( jmeno: "Petr", prijmeni: "Poškolák", rodne_cislo: "123456/1234",
        LocalDate.of( year: 2002, month: 4, dayOfMonth: 20), chlapec: true));
    System.out.println(t1.pridejNovéhoZaka( jmeno: "Aleš", prijmeni: "Premiant", rodne_cislo: "458745/8569",
        LocalDate.of( year: 2000, month: 3, dayOfMonth: 3), chlapec: true));

    System.out.println(t1.vratSeznamZaku());

    System.out.println(t1.pridejNovéhoZaka( jmeno: "Antonín", prijmeni: "Nový", rodne_cislo: "123456/1234",
        LocalDate.of( year: 2001, month: 4, dayOfMonth: 8), chlapec: true));
    System.out.println(t1.pridejNovéhoZaka( jmeno: "Alena", prijmeni: "Javová", rodne_cislo: "789457/8569",
        LocalDate.of( year: 2000, month: 2, dayOfMonth: 25), chlapec: false));

    System.out.println(t1.vratSeznamZaku());
}
```



```
Run: Main x
▶ ↑ Žákyně Vlasta Výborná byla přidána do 2. třídy
■ ↓ Žák Petr Poškolák byl přidán do 2. třídy
🔍 ↺ Žák Aleš Premiant byl přidán do 2. třídy
⚙️ *****Seznam žáků*****
🖨️ Vlasta Výborná (20. 4. 2002), 2. ročník
🗑️ Petr Poškolák (20. 4. 2002), 2. ročník
📌 Aleš Premiant (3. 3. 2000), 2. ročník
📌 *****
Chyba přidání nového žáka: Bylo zjištěno, že žák/žákyně s rodným číslem 123456/1234 již ve třídě existuje.
Žákyně Alena Javová byla přidána do 2. třídy
*****Seznam žáků*****
Vlasta Výborná (20. 4. 2002), 2. ročník
Petr Poškolák (20. 4. 2002), 2. ročník
Aleš Premiant (3. 3. 2000), 2. ročník
Alena Javová (25. 2. 2000), 2. ročník
*****
```

Pokud se pokusíme přidat jiného žáka, ovšem s již existujícím rodným číslem, z hlediska funkcionality metody můžeme na konzoli číst chybu při zápisu do seznamu. V našem případě Antonín Nový má stejné rodné číslo, jako Petr Poškolák. V jednom z objektů je chyba, neboť stejné rodné číslo u více žáků není možné použít. Alena Javová je posledním přidávaným žákem, její rodné číslo nekoliduje s žádným jiným, byla tedy úspěšně přidána do seznamu.

Již jsme schopni nového žáka do třídy přidat. Pojďme tedy zkusit stávajícího žáka ze třídy odstranit.

Metoda: Odeber žáka

Metodu **odeberZaka** budeme řešit obdobně jako metodu předchozí. Nejprve podle rodného čísla zjistíme, zda se žák ve třídě opravdu nachází. Cyklem projdeme celý seznam žáků. V případě nalezení příslušného rodného čísla následně žáka ze seznamu třídy odstraníme. Činíme tak pomocí metody `remove`. Výpis o aktuálním dění budeme opět vracet z metody ven formou textu.

Neboť atributy žáka jsou označeny jako `private`, musíme se k jejich vnitřním hodnotám dostat přes gettery. Ujistíme se tedy, že nám příslušné gettry ve třídě žáka nechybí. Pokud ano, jednoduše je doprogramujeme, abychom se vyhnuli chybovému hlášení během kompilace programu.

Pokud metoda `odeberZaka` zjistí, že žák se v seznamu třídy nenachází, pouze uživatele o této situaci informuje.

Metoda `odeberZaka` (`Trida.java`)

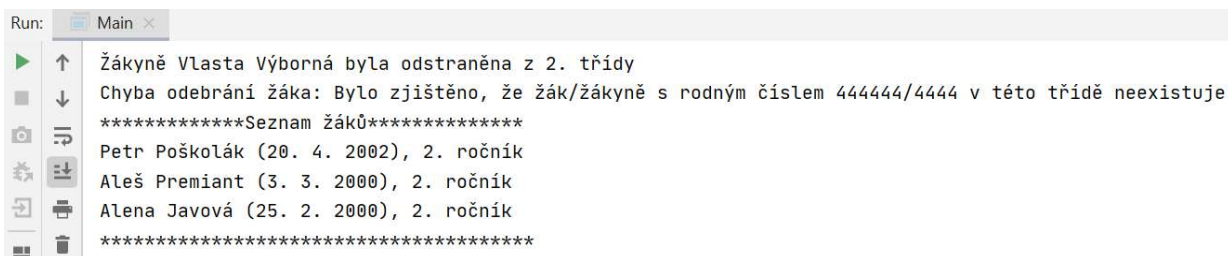
```
public String odeberZaka(String rodneCislo) {  
    if (jeZakVeTride(rodneCislo)) {  
        for (Zak z : zaci) {  
            if (z.getRodneCislo().equals(rodneCislo)) {  
                if (z.isChlapec()) {  
                    zaci.remove(z);  
                    return String.format("Žák %s %s byl odstraněn z %d. třídy",  
                        z.getJmeno(), z.getPrijemni(), z.getRocnik());  
                }  
                else {  
                    zaci.remove(z);  
                    return String.format("Žákyně %s %s byla odstraněna z %d. třídy",  
                        z.getJmeno(), z.getPrijemni(), z.getRocnik());  
                }  
            }  
        }  
    }  
    return String.format("Chyba odebrání žáka: Bylo zjištěno, že žák/žákyně s rodným číslem %s v této třídě neexistuje",  
        rodneCislo);  
}
```

Ověření metody je snadné. Do třídy přidáme 3 různé žáky. Odstraníme žáka s existujícím rodným číslem a poté s neexistujícím. Čteme výpisy na konzoli. Je zřejmé, že ze seznamu žáků byl odebrán pouze jeden prvek.

Testování metody odeberZaka (Main.java)

```
System.out.println(t1.odeberZaka( rodneCislo: "987654/3210"));
System.out.println(t1.odeberZaka( rodneCislo: "444444/4444"));
```

```
System.out.println(t1.vratSeznamZaku());
```



```
Run: Main x
Žákyně Vlasta Výborná byla odstraněna z 2. třídy
Chyba odebrání žáka: Bylo zjištěno, že žák/žákyně s rodným číslem 444444/4444 v této třídě neexistuje
*****Seznam žáků*****
Petr Poškořák (20. 4. 2002), 2. ročník
Aleš Premiant (3. 3. 2000), 2. ročník
Alena Javová (25. 2. 2000), 2. ročník
*****
```

Metoda: Zvyš ročník

Poslední metodou naprogramovanou v class Trida bude metoda **pridejRocnik**. Kód bude mít za úkol nejen upravit hodnotu vnitřního atributu ročník. Upraví rovněž ročník u každého žáka nacházejícího se v seznamu příslušné třídy. Nedávalo by smysl, kdyby třída měla v názvu 3, ale všichni její žáci by v sobě měli uložen ročník 2. Projdeme tedy celý seznam žáků a u každého z nich zavoláme setter právě na vlastnost ročník. Pokud nám setter chybí, doprogramujeme jej ve třídě Zak.java.

Metoda zvyšRocnik (Trida.java)

```
public String zvyšRocnik()
{
    ročník++;
    for(Zak z: zaci)
        z.setRocnik(ročník);
    return String.format("Třída postoupila do %d. ročníku",ročník);
}
```

Pokud ve třídě Zak metoda set schází, jednoduše ji doprogramujeme.

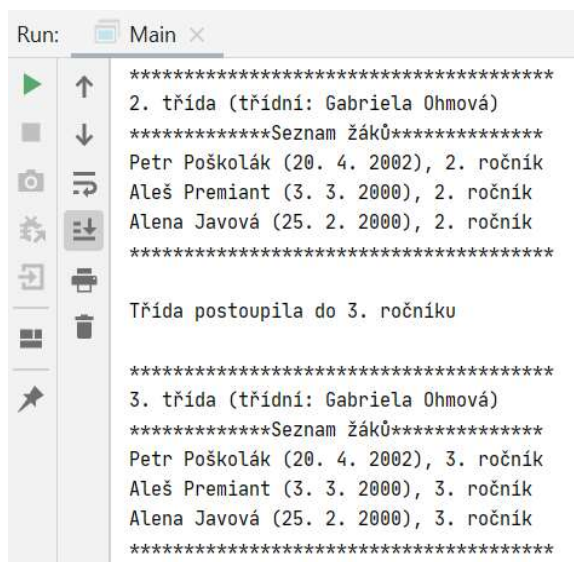
Metoda setRocnik (Zak.java)

```
public void setRocnik(byte ročník){
    this.ročník=ročník;
}
```

Poznámka: Všimněte si způsobu práce, kdy metody get či set přidáváme do programu postupně, právě když zjistíme, že jsou potřeba.

Testování metody zvysRocnik (Main.java)

```
System.out.println(t1.vratInfo());  
System.out.println(t1.zvysRocnik());  
System.out.println(t1.vratInfo());
```



```
*****  
2. třída (třídní: Gabriela Ohmová)  
*****Seznam žáků*****  
Petr Poškolák (20. 4. 2002), 2. ročník  
Aleš Premiant (3. 3. 2000), 2. ročník  
Alena Javová (25. 2. 2000), 2. ročník  
*****  
  
Třída postoupila do 3. ročníku  
  
*****  
3. třída (třídní: Gabriela Ohmová)  
*****Seznam žáků*****  
Petr Poškolák (20. 4. 2002), 3. ročník  
Aleš Premiant (3. 3. 2000), 3. ročník  
Alena Javová (25. 2. 2000), 3. ročník  
*****
```

Zadání

V class Trida naprogramujte 2 přetížené metody s názvem **vratZaka**:

- První bude vracet informace o žákovi na základě zadaného rodného čísla na vstupu. Výstupem bude text metody vratInfo() z class Zak. Případně text „Žák se zadaným rodným číslem ve třídě neexistuje“.
- Druhá metoda bude vracet informace o žákovi na základě zadané pozice (indexu) v seznamu žáků. Výstupem bude rovněž text metody vratInfo() z class Zak. Případně text: „Zadaný index neexistuje“.

Obě metody otestujte z Main.java.